
BÀI 3. GIAO TIẾP IO TỰ THIẾT KẾ

1. Mục đích

Thông qua bài thực hành này, sinh viên sẽ hiểu rõ:

- Cách thêm mô đun tự thiết kế vào thư viện trong Platform Designer.
- Cách xây dựng và kết nối các giao tiếp IO (xây dựng bằng ngôn ngữ Verilog) vào hệ thống SoPC sẵn có.
- Cách truy xuất tới các giao tiếp IO từ công cụ Nios II.

2. Phân lý thuyết

2.1. Tập project trên Quartus Prime

- Tạo project Quartus tên là “**Bai3**”. Lưu ý đường dẫn thư mục project không được có khoảng trắng. Chọn board DE2-115, chọn Family là **Cyclone IV E**, device là **EP4CE115F29C7**.
- Trong project, tạo file **switches.v** và **red_leds.v** với nội dung được mô tả như 2 đoạn code bên dưới.

```
module switches
(
    input            iClk,
    input            iReset_n,
    input            iChip_select_n,
    input            iRead_n,
    input  [31:0]    iSwitches_data,
    output reg [31:0] oSwitches_reg
);

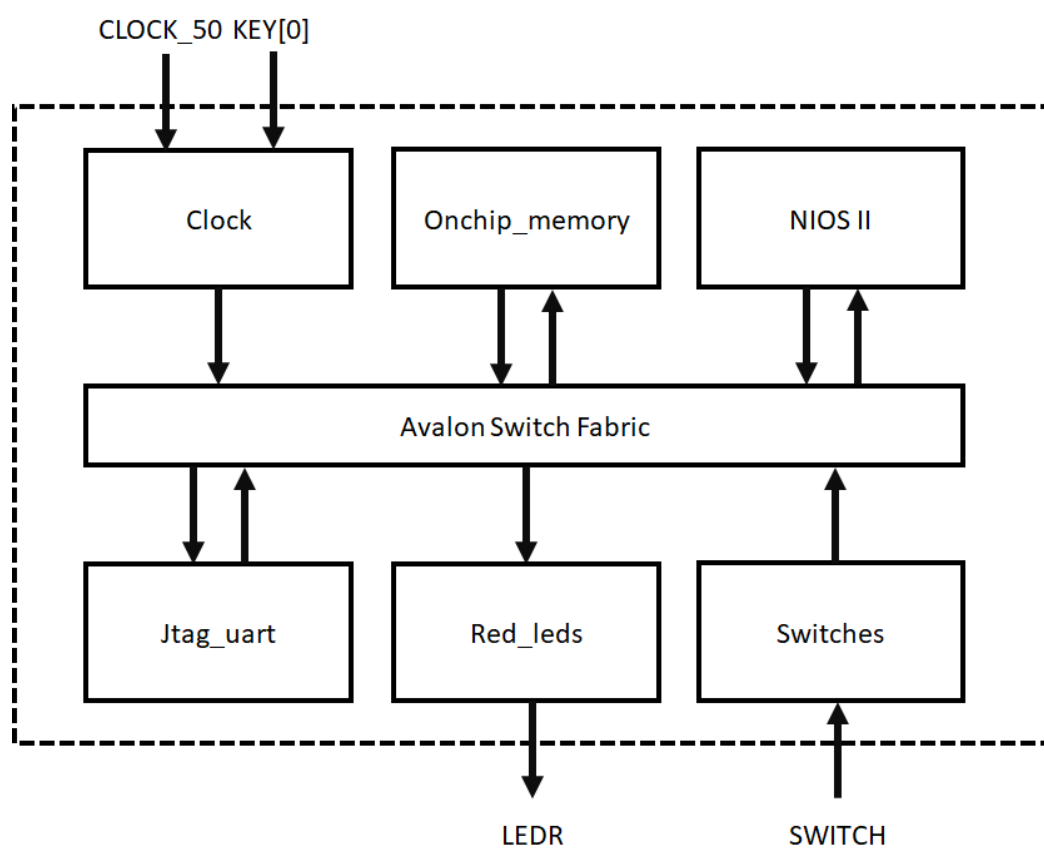
always@(posedge iClk, negedge iReset_n)
begin
    if(~iReset_n)
    begin
        oSwitches_reg <= 32'd0;
    end
    else
    begin
        if(~iChip_select_n & ~iRead_n)
        begin
            oSwitches_reg <= iSwitches_data;
        end
    end
end
```

```
end  
endmodule
```

```
module red_leds  
(  
    input          iClk,  
    input          iReset_n,  
    input          iChip_select_n,  
    input          iWrite_n,  
    input  [31:0]   iRed_leds_data,  
    output reg [31:0] oRed_leds  
);  
  
always@(posedge iClk, negedge iReset_n)  
begin  
    if(~iReset_n)  
    begin  
        oRed_leds <= 32'd0;  
    end  
    else  
    begin  
        if(~iChip_select_n & ~iWrite_n)  
        begin  
            oRed_leds <= iRed_leds_data;  
        end  
    end  
end  
endmodule
```

2.2. Tổng quan hệ thống phần cứng

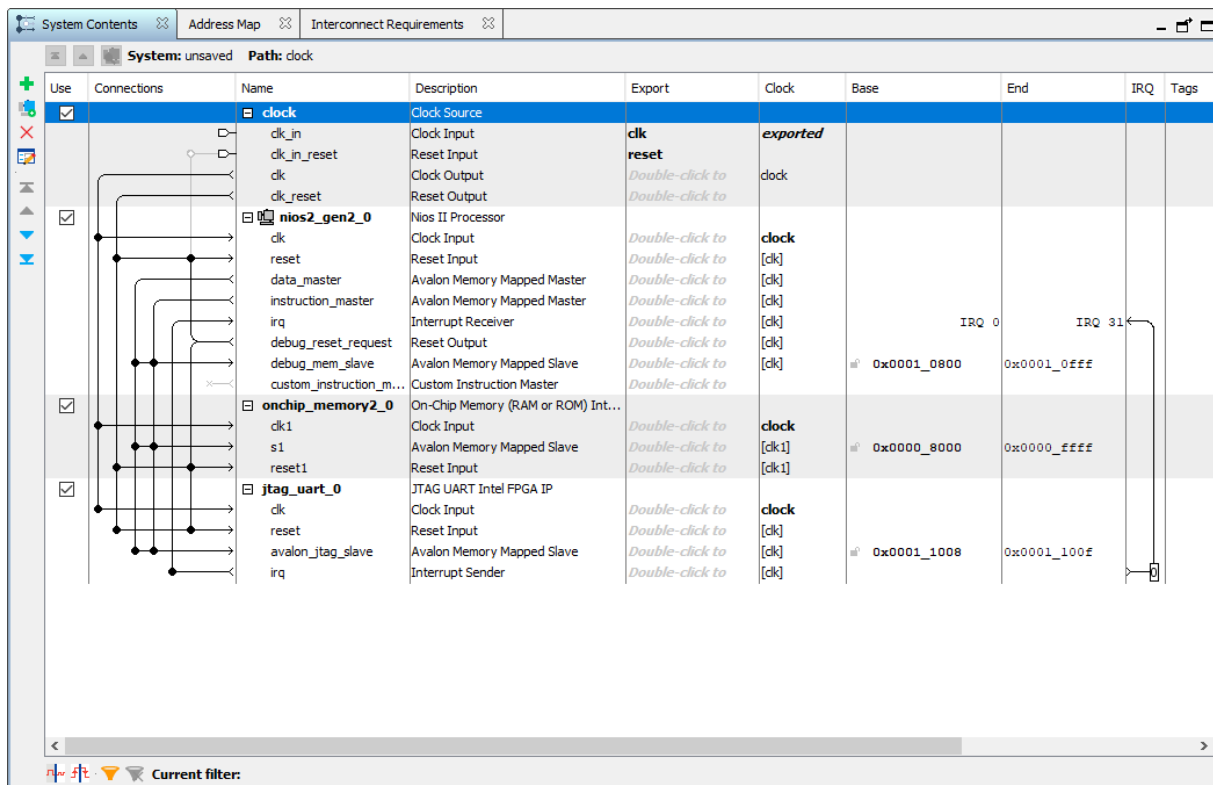
Hệ thống phần cứng được thể hiện trong hình 1 bên dưới. Trong đó mô đun Red_leds và switches là 2 mô đun giao tiếp IO tự thiết kế bằng code verilog như 2 đoạn code ở trên.



Hình 1. Tổng quan hệ thống.

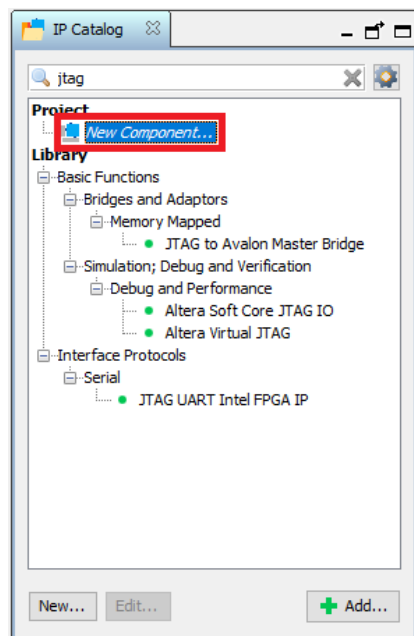
2.3. Tạo hệ thống trên Flatform Designer

- Xây dựng hệ thống như hình 2 bên dưới.



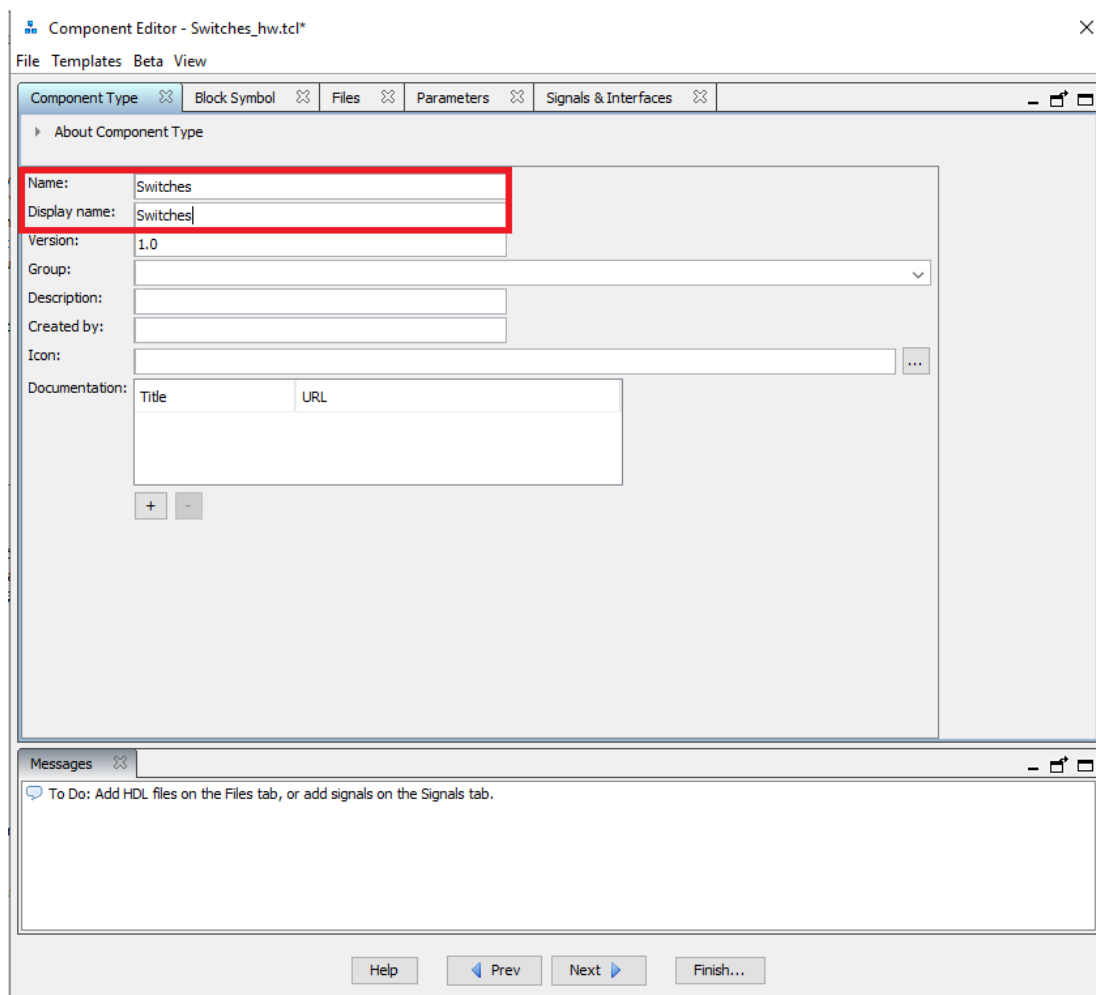
Hình 2. Hệ thống chuẩn bị ban đầu.

- Tiếp theo, tiến hành thêm mô đun switches đã được định nghĩa ở file switches.v. Tại tab thư viện – IP Catalog, ta nhấp đúp và ô New Component như hình 3.



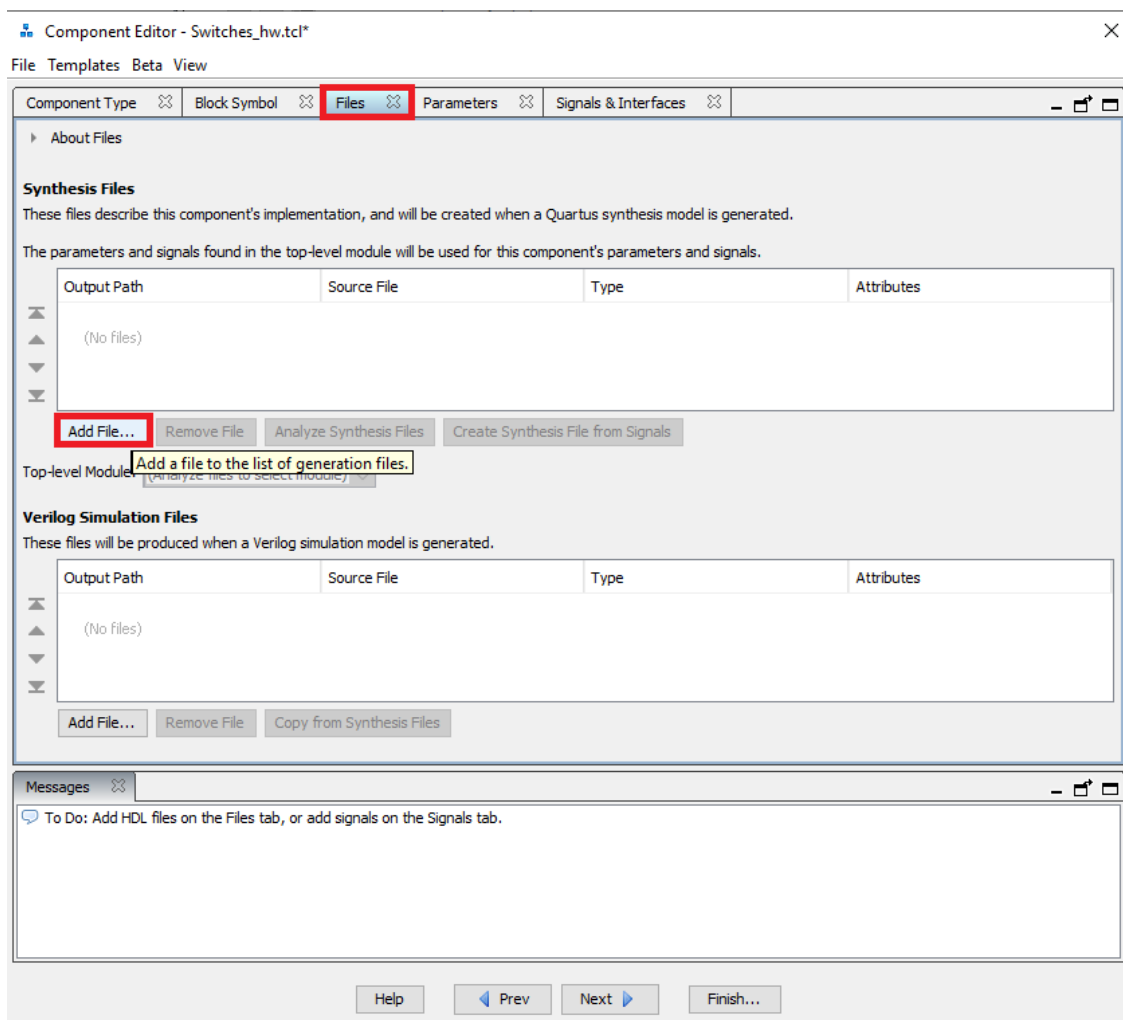
Hình 3. Tab IP Catalog.

- Sau đó, cửa sổ **Component Editor** sẽ hiện ra, tiếp tục đặt tên là **Switches** và Display name là **Switches** như hình 4.



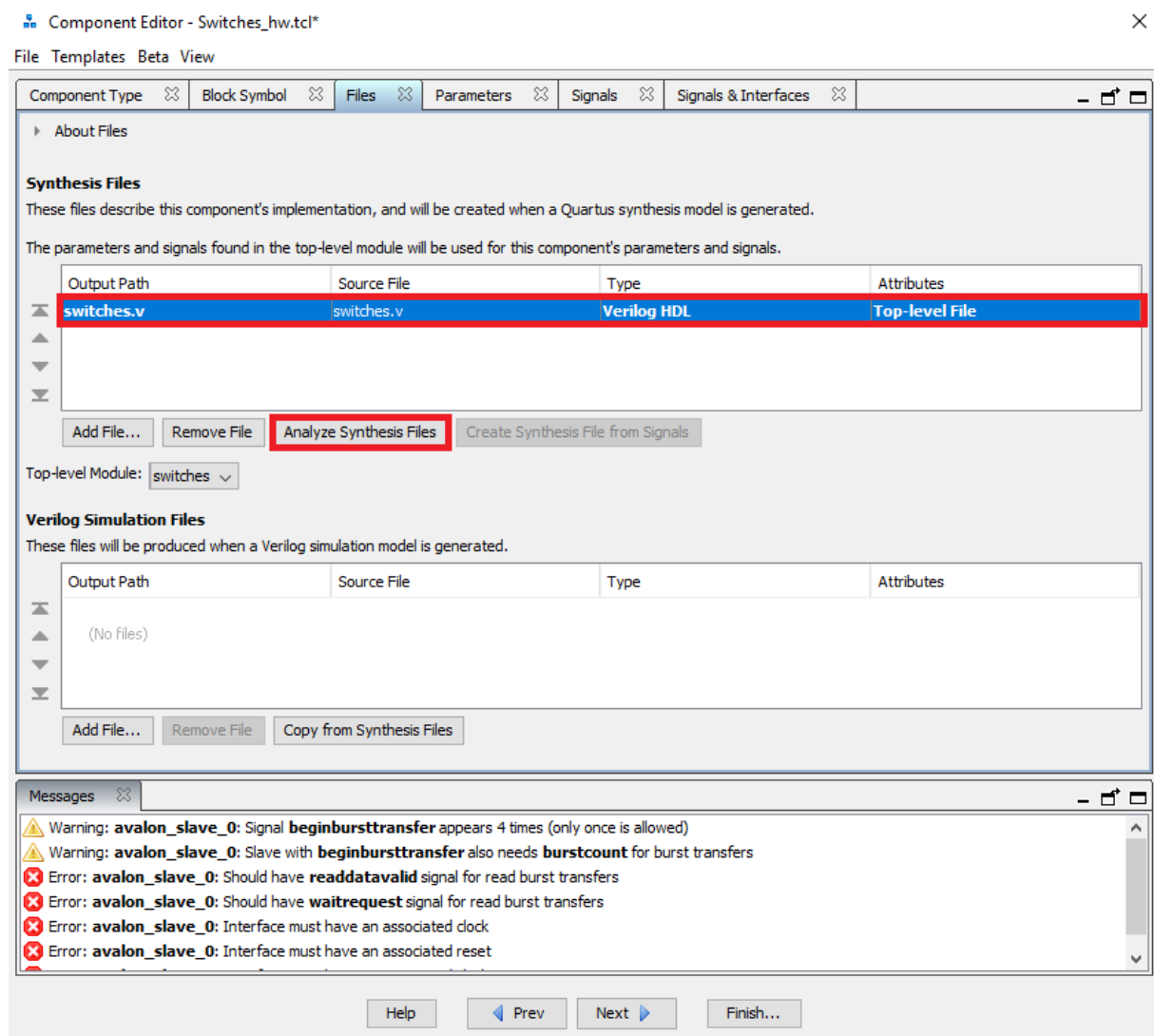
Hình 4. Tab Component Type.

- Tiếp theo, chuyển sang tab Files và chọn Add File như hình 5.



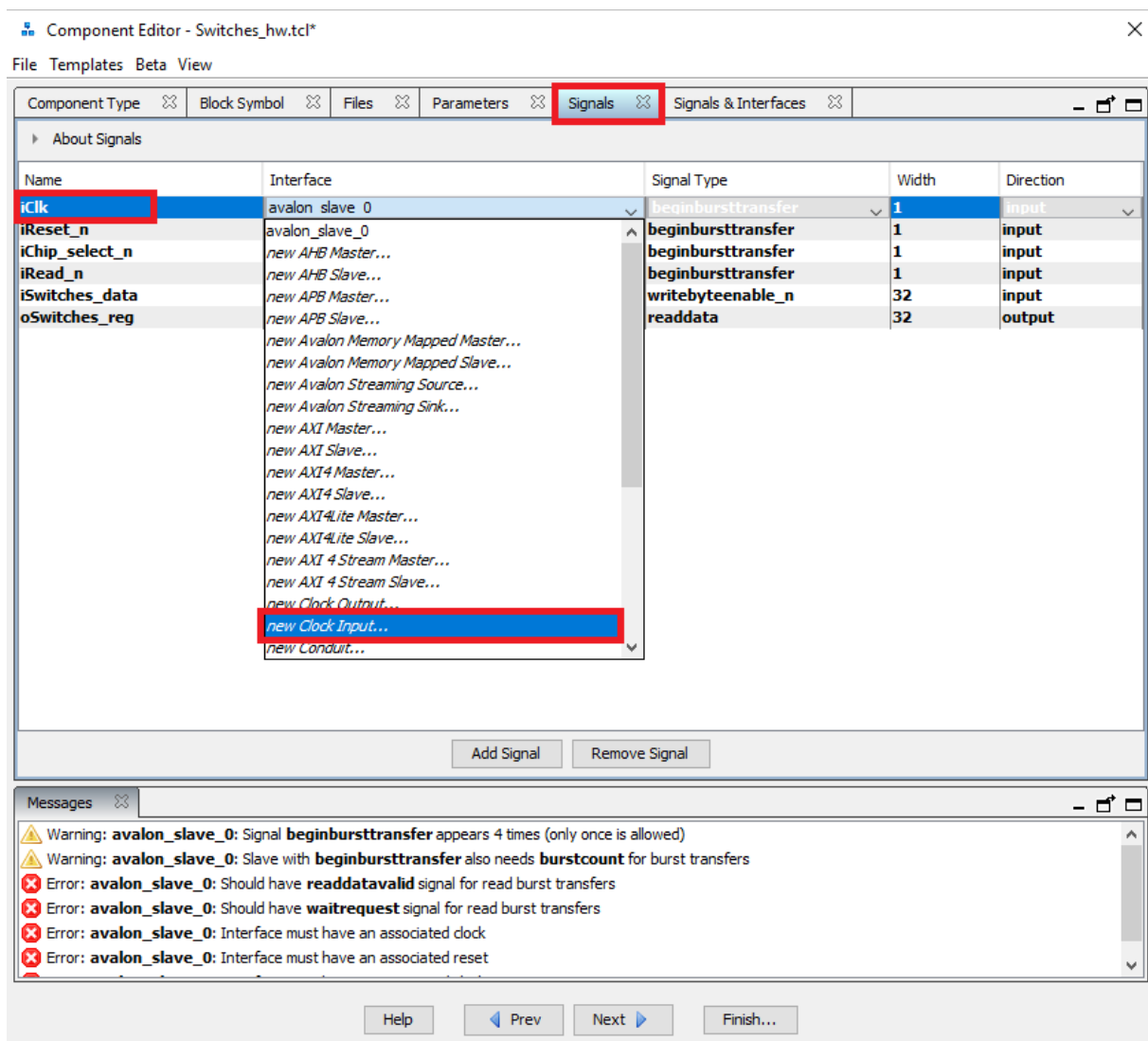
Hình 5. Tab Files.

- Sau khi thêm file hoàn tất chọn tiếp Analyze Synthesis Files như hình 6.



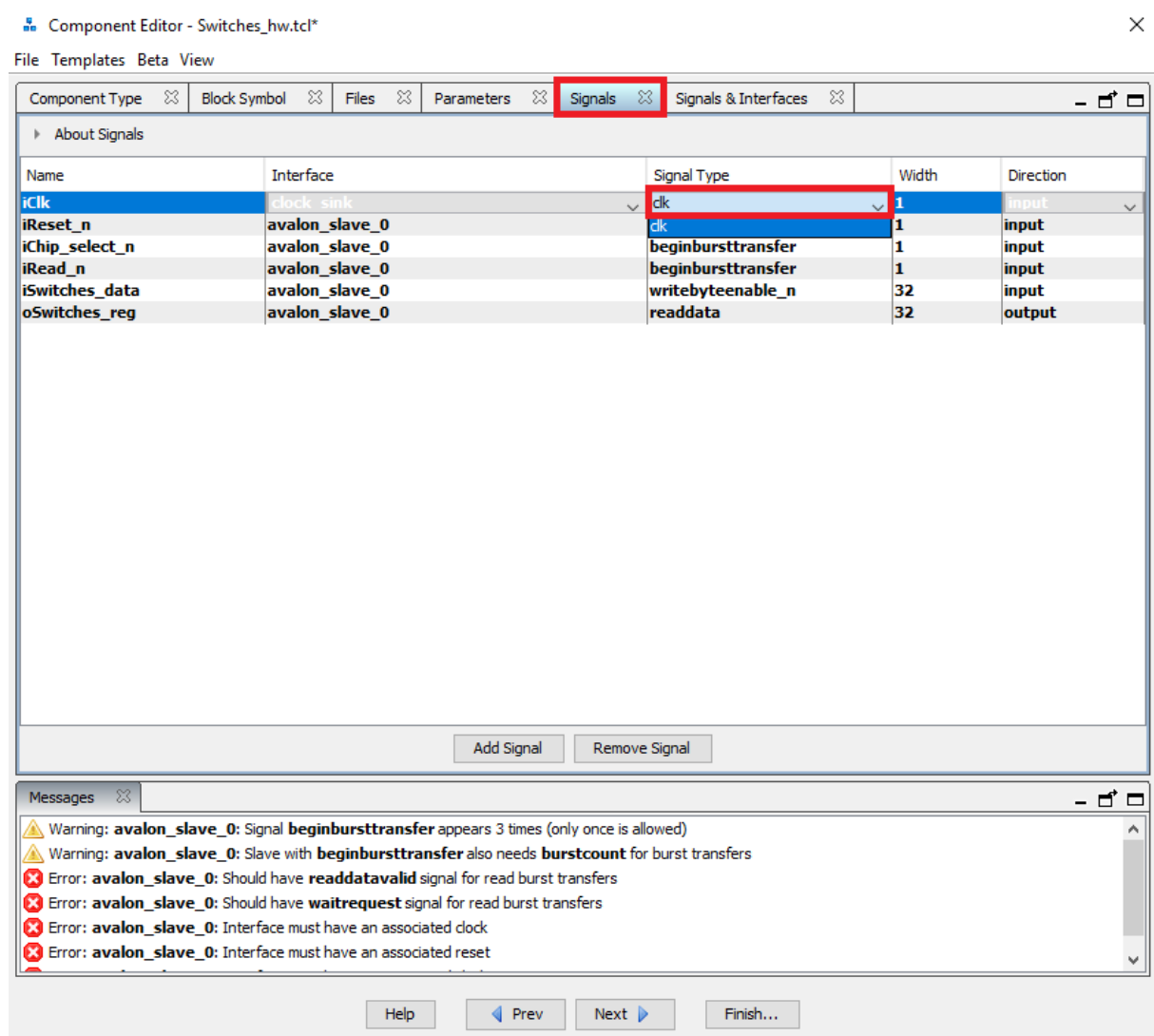
Hình 6. Thêm file và phân tích file đã thêm.

- Sau đó, chọn **View** → **Signals** để hiển thị tab **Signals**. Tiến hành cấu hình cho các tín hiệu, đầu tiên sẽ cấu hình cho tín hiệu **iClk**. Cấu hình cho tín hiệu **iClk** với interface là new Clock Input... , hình 7.



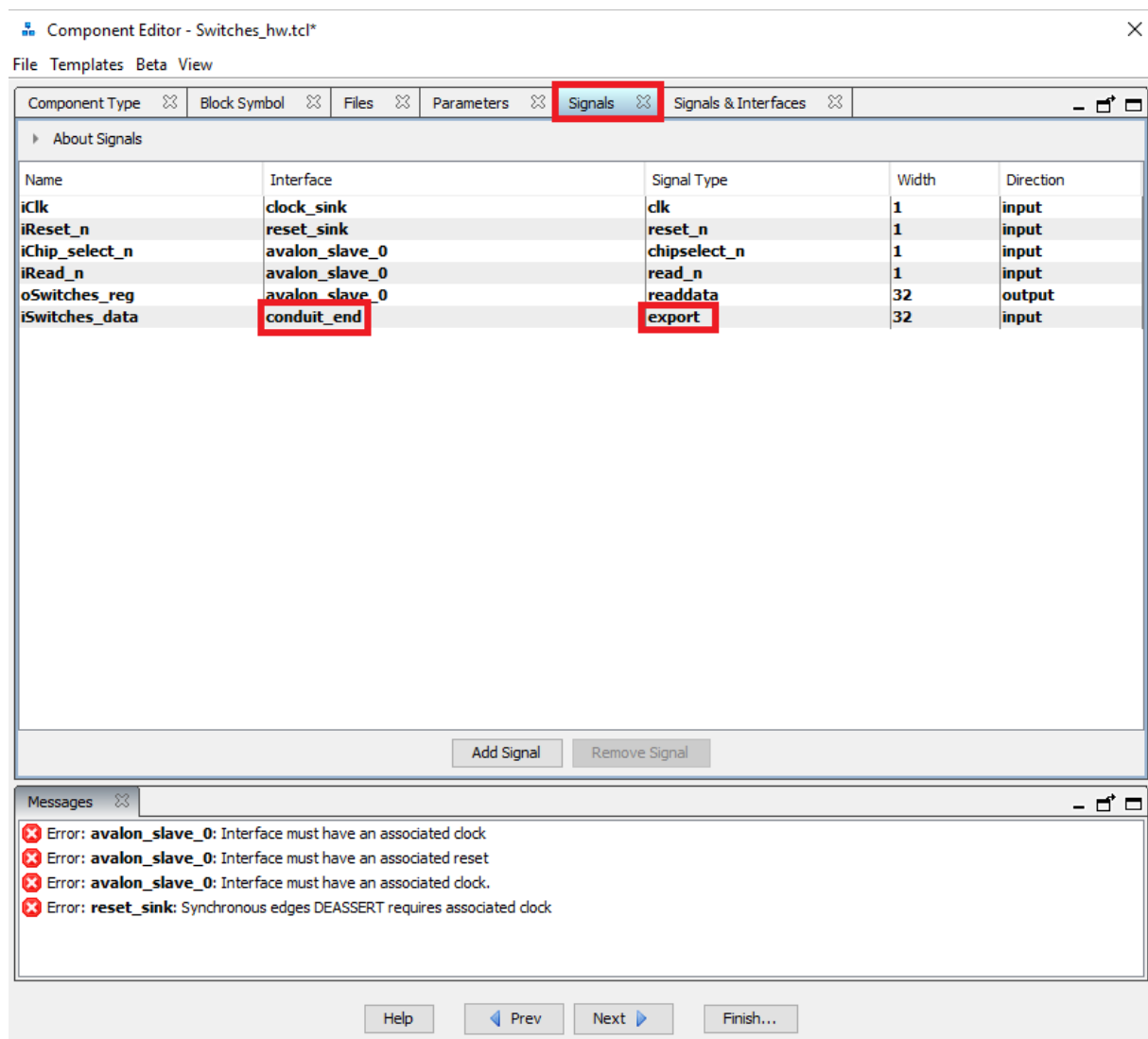
Hình 7. Tab signals.

- Tiếp tục với loại tín hiệu ở **Signal Type**, chọn là **clk**, hình 8.



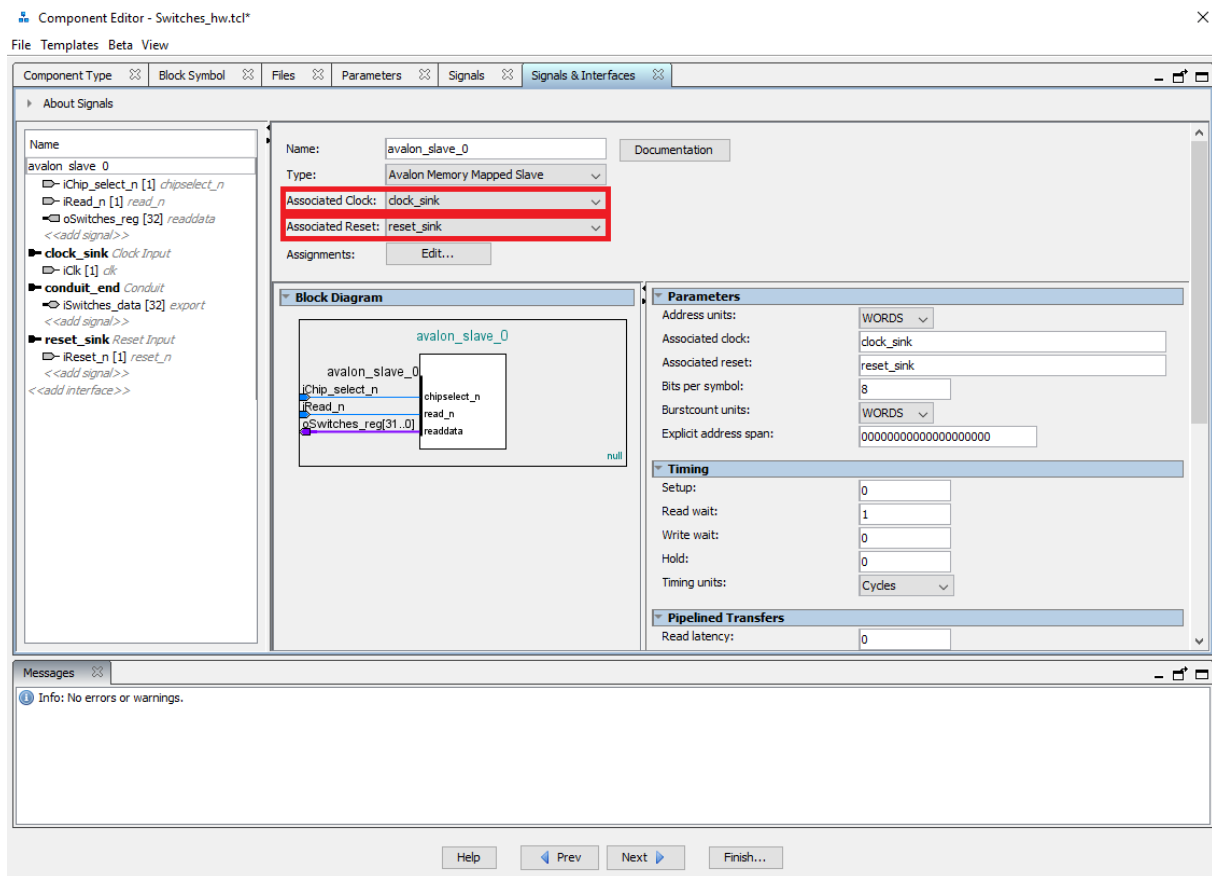
Hình 8. Cấu hình tín hiệu cho tín hiệu iClk.

- Tương tự, tiến hành cấu hình cho các tín hiệu còn lại như hình 9. Lưu ý, tín hiệu **iSwitches_data** là tín hiệu được thiết lập Interface là **conduit_end** và Signal Type gõ là **export**.



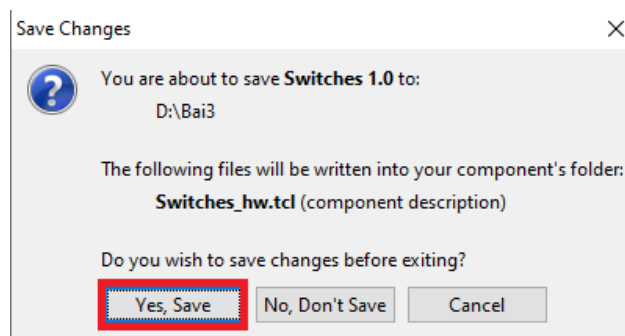
Hình 9. Cấu hình tín hiệu iSwitches_data.

- Cuối cùng chuyển sang tab **Signals & Interfaces**, tiến hành cấu hình tín hiệu Clock và Reset cho các interface. Hình 10 đại diện cho việc cấu hình tín hiệu **Clock** và **Reset** cho interface **avalon_slave_0**.



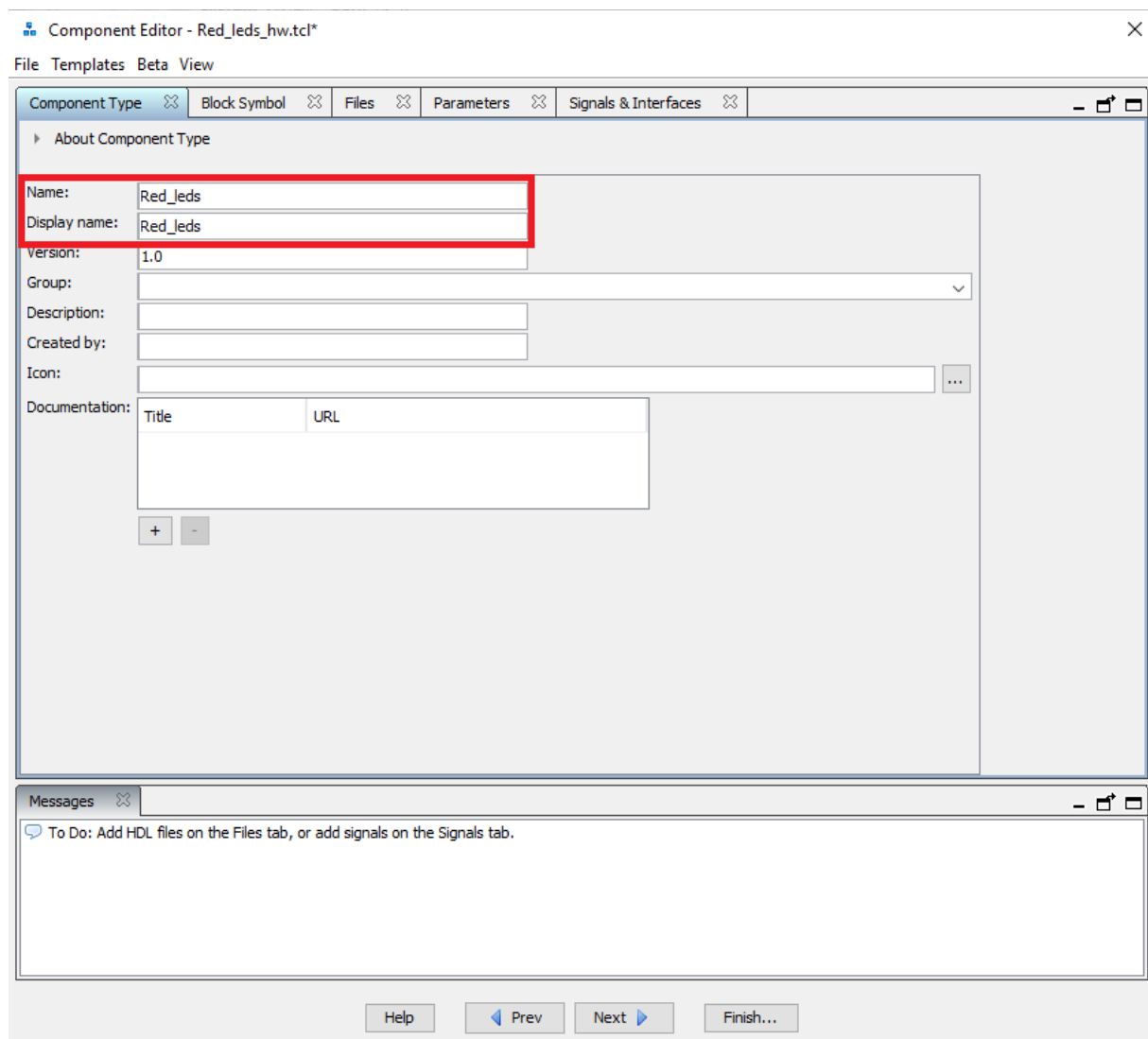
Hình 10. Tab Signals & Interfaces.

- Khi không còn lỗi, nhấn **Finish** và lưu lại để kết thúc.

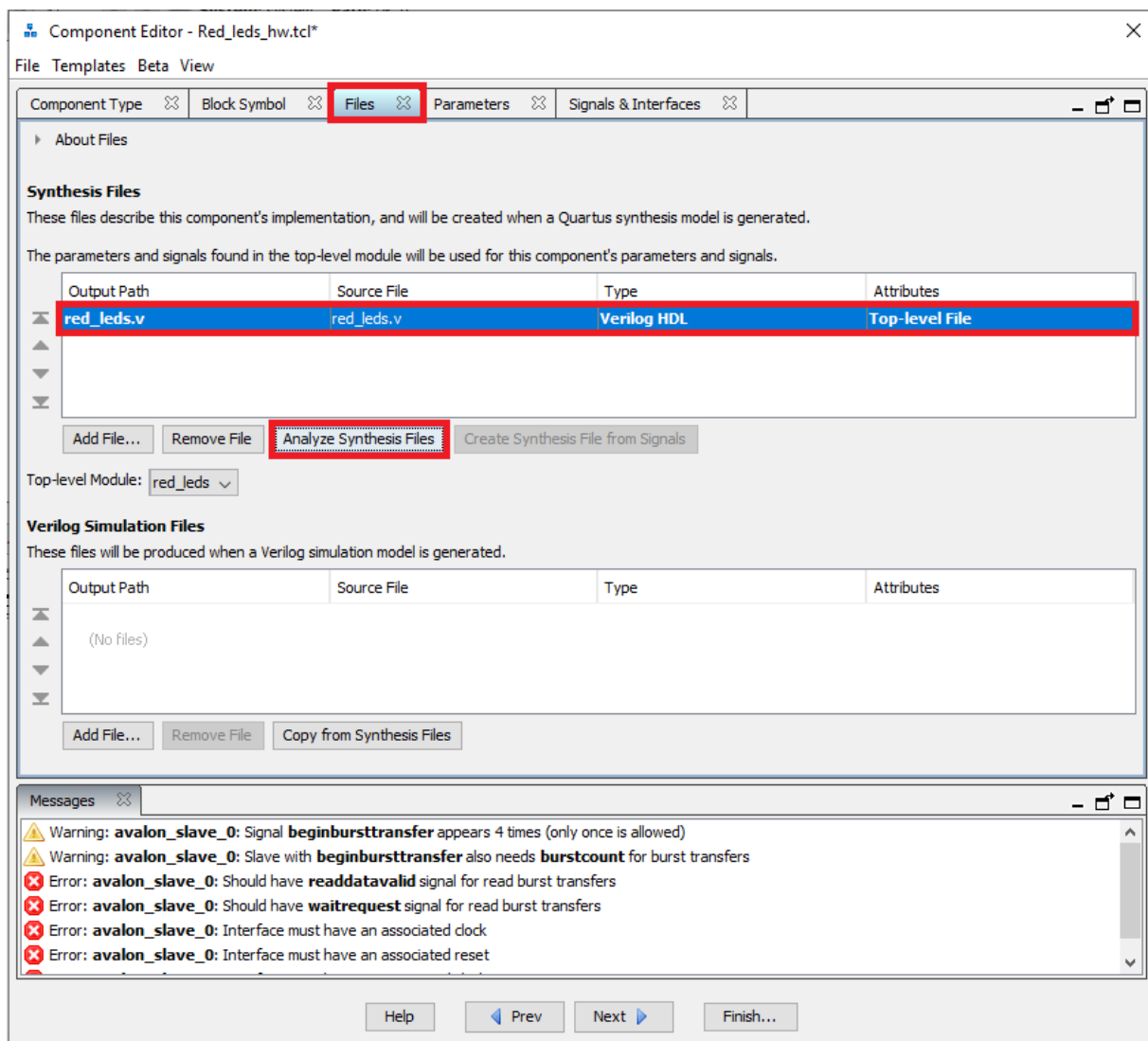


Hình 11. Lưu lại mô đun vừa thêm.

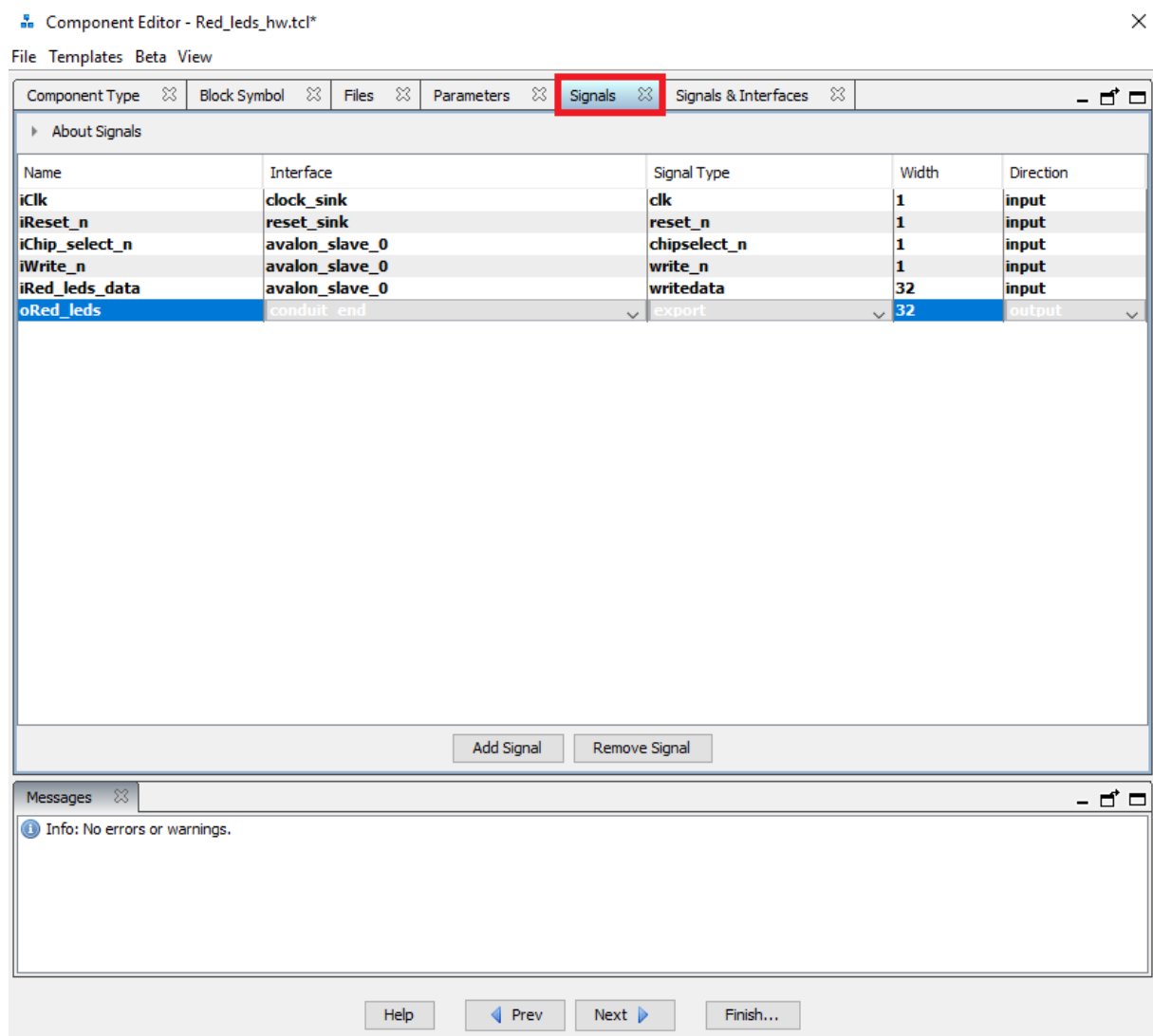
- Tương tự, ta thêm mô đun red_leds đã được định nghĩa ở file **red_leds.v**. Các bước thực hiện được thể hiện ở các hình bên dưới.



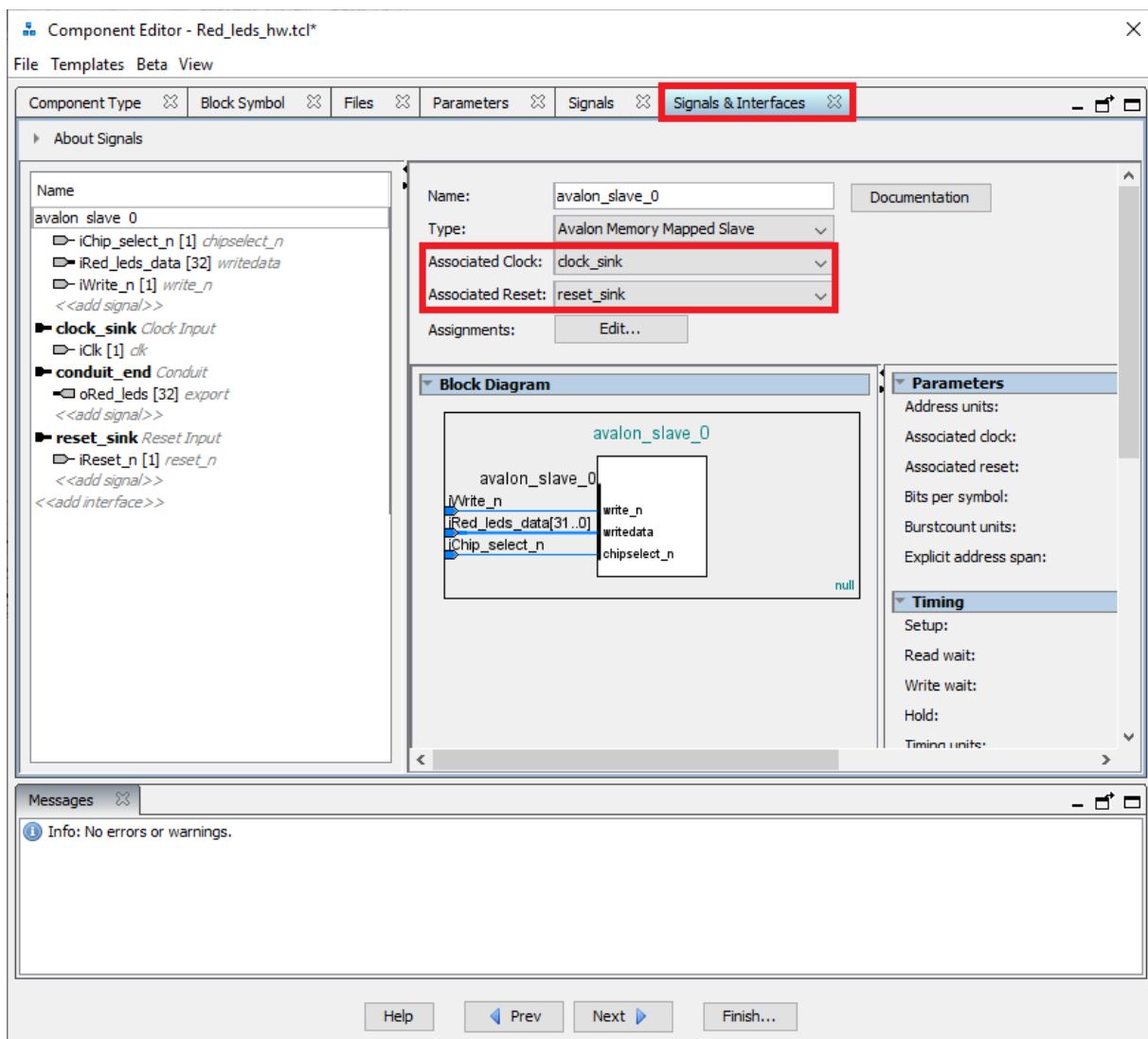
Hình 12. Tab Component Type.



Hình 13. Thêm file và phân tích ở tab Files.

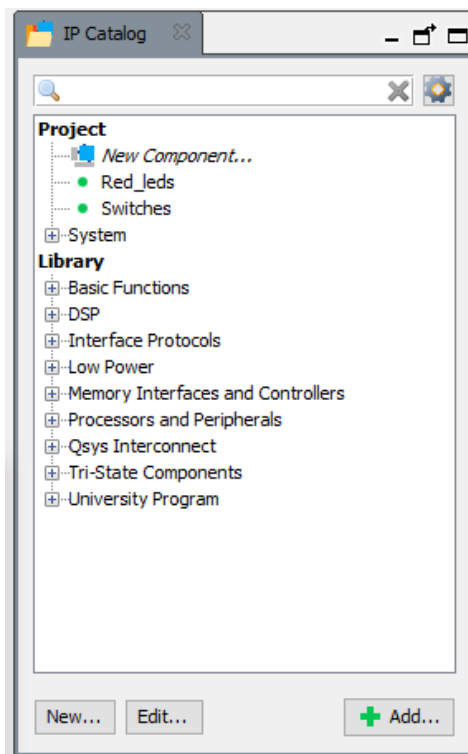


Hình 14. Cấu hình ở tab Signals.



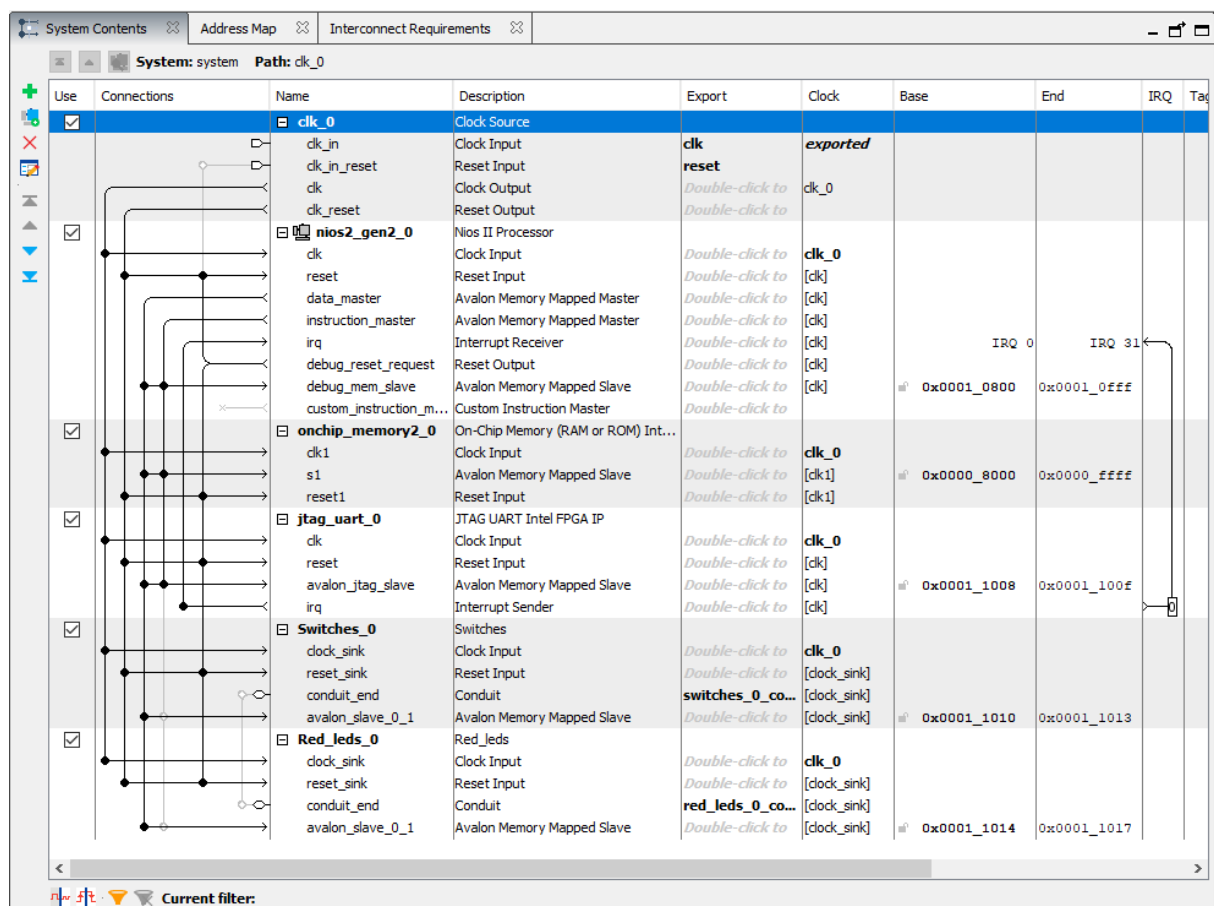
Hình 15. Cấu hình tín hiệu Clock và Reset ở tab Signals & Interfaces.

- Sau khi thêm 2 mô đun hoàn tất, có kết quả ở tab thư viện như hình 16.



Hình 16. Hai mô đun vừa thêm xuất hiện ở tab IP Catalog.

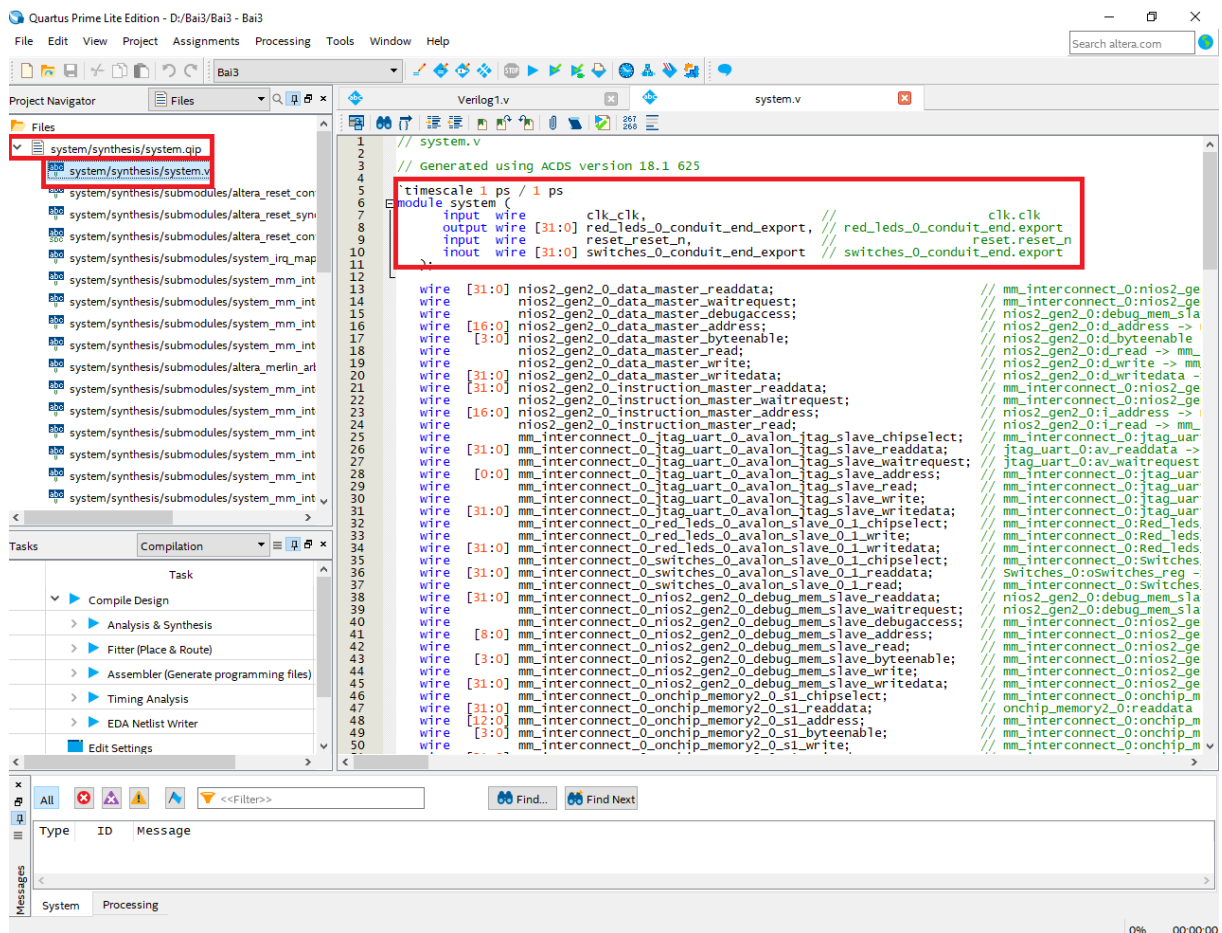
- Cuối cùng, tiến hành kết nối hệ thống như hình 17, gán địa chỉ, lưu lại và generate hệ thống với tên là system.



Hình 17. Hệ thống hoàn chỉnh.

2.4. Tích hợp hệ thống.

- Thêm file **system.qip** vào project. Mở file **system.v** để xem các tín hiệu của mô đun **system**, hình 20.



Hình 18. Thêm file `system.qip` và tín hiệu của mô đun `system`.

- Tạo file top-level là **Bai3.v** như đoạn code bên dưới. (chú ý đúng với tín hiệu file **system.v**, nếu có sự khác nhau, sinh viên sửa các tín hiệu trong file **Bai3.v** tương ứng với file **system.v**).

```

module Bai3(
    input          CLOCK_50,
    input          [0:0] KEY,
    input          [15:0] SW,
    output         [15:0] LEDR
);
system Nios_system(
    .clk_clk          (CLOCK_50),
    .reset_reset_n    (KEY[0]),
    .red_leds_0_conduit_end_export (LEDR),
    .switches_0_conduit_end_export (SW)
);

```

```
endmodule
```

- Gán chân pin cho board: **Assignments** → **Import Assignments**.
- Biên dịch phần cứng và nạp xuống board.

2.5. Lập trình phần mềm

- Tạo và đặt tên project là “**Bai3**”.
- Thêm file “**source.c**” vào project “**Bai3**”. File “**source.c**” có nội dung như đoạn code bên dưới. Build project và download phần mềm xuống board. Đổi trạng thái của Switch trên board và quan sát LED.

```
#include <stdio.h>
#include "io.h"
#include "system.h"

void main() {
    short temp;
    while (1) {
        temp = IORD(SWITCHES_0_BASE, 0);
        IOWR(RED_LEDS_0_BASE, 0, temp);
    }
}
```

BÀI TẬP CHUẨN BỊ Ở NHÀ

Bài 1. Dựa vào tài liệu tham khảo [4], vẽ dạng sóng thể hiện quá trình hoạt động của 2 file switches.v và red_leds.v

Bài 2. Viết code verilog mô tả mô đun thực hiện giải mã ra led 7 đoạn tương ứng với dữ liệu từ SW[3:0].

BÁO CÁO THỰC HÀNH

Bài 1. Đổi code ở file `source.c` để truy xuất 2 mô đun `Switches` và `Red_leds` từ Nios II thông qua con trỏ.

Bài 2. Thực hiện hệ thống SoC có nhiệm vụ giải mã led 7 đoạn tương ứng với dữ liệu từ `SW[3:0]` như code đã chuẩn bị ở bài chuẩn bị ở nhà.

TÀI LIỆU THAM KHẢO

- [1] DE2_115_User_Manual.
- [2] Embedded Peripherals IP User Guide.
- [3] Nios II Processor Reference.
- [4] Avalon Interface Specification.